

Contents

Plano — Reescrita e Transmutação de Materiais	1
1. Diagnóstico — hoje é possível reescrever um material existente?	1
2. O que implementar	2
3. Ordem de execução	3
4. Critérios de aceitação	3

Plano — Reescrita e Transmutação de Materiais

Data: 09-08-2026 Escopo: Fábrica Agêntica de Publicações — reescrever materiais já produzidos (mesmo tipo ou tipo diferente), com contrato preservado e gates obrigatórios.

1. Diagnóstico — hoje é possível reescrever um material existente?

Resposta: NÃO. A esteira só sabe **criar novo** ou **retomar/corriger**. Não existe fluxo de reescrita de material existente.

Caminho	Comportamento hoje	Evidência
/esbocar <tema> com slug existente	Cria obra paralela com sufixo - v2 — não reescreve	esbocar.md Passo 0.1; criar-livro.md Passo 0.1
Capítulos de livro (pool-capitulos.py)	capitulo_entregue (7 seções EITA + ≥3000 chars) → concluido_autonomo → sai da fila . Retomada pula entregues; --reset só zera tentativas	montar_visao em scripts/pool-capitulos.py
/criar-tcc <slug>	Reusa config_obra.json existente (só o esboço)	criar-tcc.md Passo 0.1
Playbook / lead magnet / deck / e-mails	Sobrescrita silenciosa ao rodar (write_text direto) — re-constrói da fonte e perde polimento LLM prévio (cards do Passo 3 do /criar-playbook, POLIMENTO-LLM dos e-mails)	extrair-passos-praticos.py, gerar-lead-magnet.py, gerar-deck.py, gerar-sequencia-emails.py
/criar-ebook//criar-artigo --n N	--n sobrescreve a quantidade (refatiar); pool por manifesto pula unidades com PDF/EPUB existente	criar-ebook.md Passo 1; unidade_entregue no pool
revisor-tecnico (Fase 2.5)	Única regravação existente: correções cirúrgicas em trechos defeituosos, dirigidas por relatórios de auditoria — explicitamente não reescreve capítulo inteiro	revisor-tecnico/SKILL.md Passos 1-3
Cascata entre tipos	Unidirecional e só compressão/extração (raiz → derivados). TCC→livro proibida (“nunca cascadeie uma expansão”)	esbocar.md; tipos_obra.py (derivado_de, natureza); validar_derivacao()

Consequências práticas: para regravar um capítulo é preciso apagar o arquivo à mão (o pool só reentrega o que não existe); mudar o livro inteiro exige -v2, o que **orfana** série, coleção e derivados.json;

feedback do usuário (tom, profundidade) não tem porta de entrada; e não existe conversão de um tipo para outro (ex.: TCC→livro, ebook→livro, livro→TCC).

2. O que implementar

Fase 1 — Reescrever capítulo individual (menor esforço, maior valor)

1. `pool-capitulos.py` ganha `--reescrever <n>`: move `cap_<n>.md` atual para `revisao/backups/<timestamp>/`, zera o registro do pool → capítulo volta a pendente.
2. Comando `/reescrever-capitulo <slug> <n> [motivo]`: valida obra → backup → despacha subagente-redator-capitulo com o **backup como base** (não reescreve do zero: preserva referências [N] válidas e diagramas) → reaudita só o capítulo (`validar-codigo.py --capitulo <n> --executar + gates F1/F2`) → registra no pool.
3. Skill redator-eita: seção “Modo reescrita” (contrato: manter numeração de refs, não recriar diagramas, aplicar o motivo).

Fase 2 — Reescrever a obra inteira no mesmo slug

- `/reescrever <slug> [--dossie|--sumario]`: por padrão **preserva** `config_obra.json` (tema/série/tamanho), `sumario_macro.json` e dossiê; backup completo da obra; regrava todos os capítulos via pool; re-roda Fase 2.5 (`auditar-obra.py --estrito + revisor`) e F3 (compilação + PDF); depois derivados em cascata (`playbook` → `LM/deck/e-mails`), `colecacao.py --sincronizar` e `empacotar-distribuicao.py`.
- `--dossie` = re-pesquisa; `--sumario` = re-arquiteta. Sem elas, reescrita é redação com arquitetura e fontes preservadas.

Fase 3 — Regravação guiada por feedback

- `/refinar <slug> [--capitulo N|--todos] <feedback>`: traduz feedback livre (tom, profundidade, precisão) em instruções de reescrita; base = backup; gates obrigatórios ao final; relatório de diffs (`revisao/relatorio_reescrita.json`).

Fase 4 — Salvaguardas (atravessa as Fases 1-3)

- Backup automático pré-reescrita em `revisao/backups/<ts>/` (nunca destruir o anterior).
- Gate pós-reescrita **obrigatório**: `auditar-obra.py --estrito + validar-codigo.py --executar + gates F1/F2`; commit só com 100%.
- Aviso no fluxo de derivados: re-rodar `/criar-playbook` perde polimento LLM — alerta no comando ou `--preservar-polimento` (polir só lacunas novas).
- Documentar em `AGENTS.md` (hardlink) + RTK scratchpad.

Fase 5 — Reescrita entre tipos (transmutação)

5.1. Matriz de reescrita no registro de tipos

- Novo campo `reescrever_de` em `TIP05` (`scripts/tipos_obra.py`): tipos-origem aceitos para transmutação, além do `derivado_de` de cascata. Pares iniciais:
 - `livro` ← `ebook`, `playbook`, `artigo`, `tcc` (expansão — remove a trava atual)
 - `tcc` ← `livro`, `ebook` (compressão/reframing acadêmico)
 - `ebook` ← `livro`, `tcc`, `playbook` (reescrita de tom)
 - `artigo` ← `livro`, `tcc`, `ebook`
- `python scripts/tipos_obra.py --matriz` passa a exibir as duas direções (cascata existente + transmutação).

5.2. Comando `/reescrever-como <slug-origem> --tipo <destino> [--novo-slug]`

1. Valida o par na matriz de reescrita — senão erro listando destinos possíveis.
2. **Recorte origem** → **estrutura destino**: generalizar `fatiar-obra.py` com mapa por par:

- ebook→livro: cada capítulo do ebook expande para capítulo EITA (7 seções), dossiê do livro-mãe ou novo;
 - playbook→livro: cada card (objetivo/passos/entregas/armadilhas) vira esqueleto de capítulo + pesquisa complementar;
 - artigo→livro: seções IMRaD viram capítulos expandidos;
 - livro→tcc: capítulos viram seções ACAD, citação numérica [N] → autor-data (NBR 10520);
 - tcc→artigo e livro→ebook: já existem — formalizar no mesmo comando.
3. **Produtor por destino** (reuso): redator-eita (livro), redator-academico (tcc/artigo), redator-ebook (ebook) — cada skill ganha seção “Modo transmutação”: preservar referências reais, diagramas reutilizáveis e motivo condutor da origem.
 4. **Registro**: derivados.json da origem (carregar_derivados/gravar_derivados) + colecao.py --sincronizar; config_obra.json do destino grava slug_origem (rastreadibilidade).
 5. **Gates do DESTINO** obrigatórios: livro → auditar-obra.py --estrito + gates F1/F2 + validar-codigo.py --executar; tcc → validar-abnt-tcc.py; ebook → EBOOK-LEN; artigo → auditoria de artigo.

5.3. Regra de derivação (AGENTS.md, hardlink)

- Substituir “nunca cascadeie expansão”: compressão/extração continuam baratas; **expansões permitidas** com custo de geração declarado pelo custo_llm (aviso no comando, pool em lotes de 4).

5.4. Salvaguardas (herda Fase 4)

- Backup da origem antes de transmutar; slug destino usa sufixo_slug (ex.: --liv, --tcc); **origem nunca é destruída**; relatório de transmutação; testes de matriz (pares válidos/inválidos), recorte e registro em derivados.json.

3. Ordem de execução

1. Fases 1-2 (reescrita mesmo tipo: capítulo → obra) — reusam pool, gates F1/F2 e skills existentes.
2. Fase 5 (transmutação) — reusa o motor de reescrita das Fases 1-2.
3. Fase 3 (refinamento guiado por feedback) — reusa as mesmas skills.
4. Fase 4 (salvaguardas) — atravessa todas as fases.

4. Critérios de aceitação

- ☐ /reescrever-capitulo <slug> <n> regrava um capítulo preservando refs/diagramas e passa nos gates.
- ☐ /reescrever <slug> regrava a obra inteira no mesmo slug sem orfanar série/coleção/derivados.
- ☐ /refinar <slug> <feedback> aplica feedback com relatório de diffs.
- ☐ /reescrever-como <slug> --tipo <destino> transmuta com gates do destino verdes (ou nao_verificado legítimo).
- ☐ Backup automático em revisao/backups/ antes de qualquer reescrita; origem nunca destruída.
- ☐ Suíte de testes 100% verde antes de commit/push.